

Compte-rendu TP9

C - Application producteur / consommateur

C1

Sur le terminal TTY[0] on devrait voir les messages:\

```
transmitted value : 1 temporisation = 100
```

...

```
transmitted value : 49 temporisation = 100
```

Sur le terminal TTY[1] on devrait voir les messages:\

```
received value : 1 temporisation = 100
```

...

```
received value : 49 temporisation = 100
```

On ne sait pas si le TTY[1] va afficher de 0 à 48 ou de 1 à 49 ou autre chose.

Comme le TTY[0] et le TTY[1] ont les mêmes délais, cela dépend de si le TTY[0] est plus rapide ou plus lent que le TTY[1].

C2

Question C2 : Complétez le code assembleur contenu dans le fichier reset.s pour que les tâches producer et consumer soient lancées sur les processeurs 0 et 1 respectivement.

C3

- PRODUCER_DELAY=100, CONSUMER_DELAY=100: comme les délais sont égaux, les valeurs affichées sont des fois les mêmes 2 fois de suite pour le consumer. Pour le producer, les valeurs sont la suite de 0 à 49. Ceci est dû au fait que des fois le consumer est plus rapide que le producer ou inversement.
- PRODUCER_DELAY = 10, CONSUMER_DELAY = 1000: Comme le producer est plus rapide que le consumer, les valeurs ne sont jamais 2 fois les mêmes pour le consumer, sauf lorsque le producer atteint 49, le consumer n'affiche alors que 49.
Le consumer saute également certaines valeurs qu'il n'a pas eu le temps d'afficher car le producer est plus rapide.
- PRODUCER_DELAY = 1000, CONSUMER_DELAY = 10: Le consumer est plus rapide que le producer, certaines valeurs sont donc affichées plusieurs fois à la suite et le consumer finit son programme avant le producer car sa dernière valeur reçue est 37 alors que le producer va jusqu'à 49.

On remarque donc que en changeant le délai les valeurs nous montrent lorsque un des deux processeurs est plus rapide que l'autre par le fait qu'ils sont désynchronisés par leur délai.

Il est donc nécessaire d'implémenter un mécanisme de synchronisation s'il l'on veut les synchroniser.

D - Synchronisation par bascule SET/RESET

D1

Il n'y a pas de risque d'incohérence liée aux accès concurrents à la variable *sync* car celle-ci ne peut pas avoir d'accès concurrents en écriture. De plus, lorsque l'un a modifié la variable *sync*, il ne peut pas la modifier tant que l'autre ne l'a pas modifié, il ne peut donc y avoir de problème d'incohérence.

D2

Avec synchronisation:

- 530 236 cycles, PRODUCER_DELAY=100, CONSUMER_DELAY=100
- 476 290 cycles, PRODUCER_DELAY=10, CONSUMER_DELAY=100
- 476 245 cycles, PRODUCER_DELAY=100, CONSUMER_DELAY=10

Sans synchronisation:

- 272 833 cycles, PRODUCER_DELAY=100, CONSUMER_DELAY=100

D3

La cachabilité des variables *sync* et *buf* introduit un risque de dysfonctionnement car si par exemple PROC0 LIT *sync* et l'enregistre dans son cache, si PROC1 écrit *sync*, PROC0 lors de sa prochaine lecture fera un HIT sur le cache et PROC0 ne verra pas la nouvelle valeur écrite par PROC1.

Il faut donc activer le mécanisme de SNOOP afin d'invalider les lignes de caches incohérentes.

D4

En désactivant le mécanisme de Snoop, comme le processeur 0 a enregistré la première valeur de Snoop lue dans son cache, celui-ci attend qu'elle passe à 1 mais ce ne sera jamais le cas car une lecture sera suivie d'un HIT sur le cache, empêchant de voir la nouvelle valeur de *sync* écrite par le processeur 1.

En l'occurrence le processeur 1 ne pourra même pas modifier la variable *sync* car il attend que le processeur 0 la modifie, mais le même problème de cache est présent dans le processeur 1.

Désactiver le mécanisme de Snoop met donc les deux programmes dans une boucle infinie.

D5

Le coût matériel d'un mécanisme comme Snoop est qu'il doit constamment vérifier les lignes de caches, il doit donc être relié à chaque cache de chaque processeur.

E - Problèmes de synchronisation liés au réordonnancement des instructions

E1

Pour assurer l'ordre d'exécution des instructions, on place l'appel `__sync_synchronize()` juste après l'écriture de BUF, val et sync.

E2

Le code binaire a introduit l'instruction *sync* dans le code de *producer()* et de *consumer()*.
Le temps d'exécution ne change pas, le programme se termine au bout de 530 026 cycles.

F - Synchronisation par FIFO logicielle

F1

Le canal de communication est une structure déclarée comme une variable globale car elle doit être accessible par tous les coeurs.

Les différents champs de la structure sont:

- *buf* : buffer contenant les données de la fifo
- *ptw* : pointeur d'écriture
- *ptr* : pointeur de lecture
- *sts* : status, nombre de données dans la fifo
- *depth* : profondeur, taille du buffer de donnée de la fifo
- *lock* : verrou informant si la fifo est libre ou en cours d'utilisation

F2

Il est préférable d'utiliser un verrou à ticket afin d'assurer une allocation équitable de la fifo, pour éviter le phénomène de "famine" et faire en sorte que chaque processeur ait autant accès à la fifo qu'un autre processeur.

On veut éviter que la fifo soit monopolisée par les processeurs ayant les tâches les plus rapides.

F3

L'argument des deux fonction *lock_acquire* et *lock_release* est *plock* qui est une structure de verrou sous forme de ticket.

F4

lock_acquire tire un numéro de ticket qui est libre avec la fonction *atomic_increment*, puis attend que celui-ci soit égal au ticket courant.

La fonction *atomic_increment* est écrite en assembleur afin d'effectuer une réservation LL/SC pour assurer la cohérence mémoire de la prise de ticket.

atomic_increment récupère *ptr* dans \$10 et *increment* dans \$22.

Puis réserve l'espace 0(*ptr*) si c'est possible.

Ajoute *increment* à \$12 qui doit correspondre à l'ancienne valeur de 0(*ptr*) et enregistre le résultat dans \$13.

Fait un store conditionnel de la valeur incrémentée dans 0(*ptr*), puis recommence si ça n'a pas marché.

Enfin, lorsque cela a marché, met l'ancienne valeur dans *value* et la retourne.

La fonction renvoie donc l'ancienne valeur pointée par *ptr* et incrémente celle-ci de la valeur *increment*.

Dans le cas de *lock_acquire*, *atomic_increment* incrémente la valeur du prochain ticket libre de façon atomique afin d'assurer la cohérence mémoire et de gérer les accès concurrents à cette variable.

F5

Pour libérer le verrou, *lock_release* incrémente la valeur du ticket courant.

Cette fonction n'a pas besoin d'être écrite en assembleur car la cohérence est déjà assurée par le mécanisme de prise de verrou.

En effet, il ne peut y avoir d'écriture concurrente sur cette variable car seule le programme ayant obtenu un verrou, dont la cohérence est assurée, peut libérer ce verrou.

F6

Si une tâche ne peut effectuer son transfert parce que la FIFO est pleine, elle doit impérativement libérer le verrou sous risque de provoquer un deadlock qui empêcherait tout autre programme de prendre le verrou de la fifo dont le lecteur.

F7

fifo_write:

```
fifo->buf[fifo->ptw] = *val;\n
fifo->ptw = (fifo->ptw + 1) % fifo->depth;\n
fifo->sts += 1;\n
lock_release((lock_t *) (&fifo->lock));\n
done = 1;
```

fifo_read:

```
*val = fifo->buf[fifo->ptr];\n
fifo->ptr = (fifo->ptr + 1) % fifo->depth;\n
fifo->sts -= 1;\n
lock_release((lock_t *) (&fifo->lock));\n
done = 1;
```

F8

DEPTH	1	2	4	8
Producer	611016	572782	526714	562928
Consumer	615322	565690	556412	586661

On peut conclure que la profondeur de la fifo a une influence sur le nombre de cycles. Si elle est trop petite, celle-ci peut faire attendre le producteur ou le consommateur.

F9

Pour 100 itérations:

- Le producer se finit au bout de 262 359 cycles.

- Le consumer se finit au bout de 262 479 cycles.

On a donc un coût moyen par itération de 262 cycles pour le producer et le consumer, ce qui correspond au coût de communication.

G - Application logicielle multi-tâches

G1

Pour lancer six processeurs différents, on doit modifier l'accès au main des 4 derniers processeurs:

```
# jump to main in user mode: main[2]
la    $26, seg_data_base\
lw     $26, 8($26)           # read main[2]\
mtc0   $26, $14             # write it in EPC register
```

Voici le résultat de la simulation:

```

tty_0 (sur gallus) x
transmitted value : 32    temporisation = 697
transmitted value : 33    temporisation = 324
transmitted value : 34    temporisation = 585
transmitted value : 35    temporisation = 709
transmitted value : 36    temporisation = 237
transmitted value : 37    temporisation = 640
transmitted value : 38    temporisation = 765
transmitted value : 39    temporisation = 887
transmitted value : 40    temporisation = 141
transmitted value : 41    temporisation = 98
transmitted value : 42    temporisation = 37
transmitted value : 43    temporisation = 322
transmitted value : 44    temporisation = 689
transmitted value : 45    temporisation = 13
transmitted value : 46    temporisation = 890
transmitted value : 47    temporisation = 808
transmitted value : 48    temporisation = 832
transmitted value : 49    temporisation = 649

*** Completing producer at cycle 355615 ***

!!! Exit Processor 0x000 !!!
[]

tty_1 (sur gallus) x
received value : 33    temporisation = 527
received value : 34    temporisation = 613
received value : 35    temporisation = 64
received value : 36    temporisation = 337
received value : 37    temporisation = 501
received value : 38    temporisation = 893
received value : 39    temporisation = 154
received value : 41    temporisation = 185
received value : 40    temporisation = 407
received value : 43    temporisation = 442
received value : 42    temporisation = 196
received value : 44    temporisation = 349
received value : 45    temporisation = 845
received value : 46    temporisation = 837
received value : 47    temporisation = 494
received value : 49    temporisation = 704
received value : 48    temporisation = 698
well received all tokens

*** Completing consumer at cycle 376926 ***

!!! Exit Processor 0x001 !!!
[]

tty_2 (sur gallus) x
*** Starting task consumer on processor 2 ***

received value : 2    temporisation = 935
received value : 5    temporisation = 576
received value : 10   temporisation = 170
received value : 17   temporisation = 286
received value : 20   temporisation = 1001
received value : 24   temporisation = 396
received value : 28   temporisation = 528
received value : 33   temporisation = 345
received value : 35   temporisation = 960
received value : 39   temporisation = 409
received value : 42   temporisation = 918
received value : 47   temporisation = 832
[]

tty_3 (sur gallus) x
*** Starting task consumer on processor 3 ***

received value : 1    temporisation = 945
received value : 11   temporisation = 273
received value : 16   temporisation = 988
received value : 22   temporisation = 646
received value : 27   temporisation = 691
received value : 32   temporisation = 606
received value : 36   temporisation = 552
received value : 41   temporisation = 0
received value : 44   temporisation = 148
received value : 46   temporisation = 282
[]

tty_4 (sur gallus) x
*** Starting task consumer on processor 4 ***

received value : 0    temporisation = 460
received value : 4    temporisation = 64
received value : 6    temporisation = 453
received value : 8    temporisation = 941
received value : 13   temporisation = 292
received value : 15   temporisation = 549
received value : 18   temporisation = 945
received value : 21   temporisation = 157
received value : 23   temporisation = 797
received value : 26   temporisation = 398
received value : 29   temporisation = 77
received value : 31   temporisation = 12
received value : 38   temporisation = 640
received value : 45   temporisation = 18
received value : 49   temporisation = 77
[]

tty_5 (sur gallus) x
*** Starting task consumer on processor 5 ***

received value : 3    temporisation = 909
received value : 7    temporisation = 256
received value : 9    temporisation = 881
received value : 12   temporisation = 305
received value : 14   temporisation = 1006
received value : 19   temporisation = 360
received value : 25   temporisation = 283
received value : 30   temporisation = 649
received value : 34   temporisation = 70
received value : 37   temporisation = 307
received value : 40   temporisation = 839
received value : 43   temporisation = 8
received value : 48   temporisation = 733
[]

```

G2

On préfère utiliser un verrou indépendant pour chaque canaux afin de paralléliser au maximum la communication. Si on avait un verrou pour tous les canaux, alors si un producer ou consumer interagit avec un canal, il bloquerait l'accès à tous les autres canaux en même temps.